Project 2 Report:
Computing the dot product of two vectors

12411723 杨嘉乐

目录

1 实验概述与环境配置 **4**
1.1 测试目标与范围 4
1.2 软硬件测试环境 4
1.3 核心实现逻辑与高精度计时机制 4

2 复杂度与线性关系验证 **5**
2.1 向量规模与执行时间的正相关性分析 5
2.2 硬件缓存层级对理论复杂度的影响 6
2.3 基于 VTune 的底层微架构瓶颈定位 7
2.3.1 指令流水线执行效率分析 7
2.3.2 存储层级瓶颈定位与访存特征 7
2.3.3 向量化效率与计算密度分析 8

3 性能对比 **12**
3.1 测试结果总览 12
3.2 基准执行耗时对比 14
3.3 基准状态下的内存限制与执行效率分析 16
3.3.1 C 语言 (`-O0`) 禁用寄存器分配的访存开销 16
3.3.2 C 语言数据位宽对缓存行填充率的影响 16
3.4 Java JIT 解释与分层编译机制 17
3.4.1 分层编译架构与运行时 Profiling 17
3.4.2 回边计数器与栈上替换 (OSR) 触发机制 17
3.4.3 C2 编译器的底层微架构映射 18
3.4.4 基于字节码的初始执行状态验证 18

4 C 语言编译优化选项与性能影响 **19**
4.1 各编译优化选项耗时对比 ($N = 10^8$) 19
4.2 基础与进阶编译优化 ($-O1$ 至 $-O3$) 20

4.2.1 $-O1$: 寄存器分配使能 20
4.2.2 $-O2$: 指令调度与对齐优化 20
4.2.3 $-O3$: 自动向量化 (SIMD) 激活 20
4.2.4 $-Os$: 代码体积与执行吞吐量的权衡 20
4.3 浮点数学模型优化 20
4.3.1 $-O3$ 结合 $-Ofast$ / $-ffast-math$ 20
4.3.2 $-fno-finite-math-only$ 的保守性表现 20
4.4 特定硬件指令集使能 21
4.4.1 $-march=native$: AVX2 256-bit 寄存器激活 21
4.4.2 $-mfma$: 硬件级融合乘加计算 21
4.5 链接时优化 (LTO) 与循环展开 21
4.6 最优编译组合性能评估 21

5 Java 虚拟机 (JVM) 运行参数与内存分析 21
5.1 大规模计算下的各参数组合耗时对比 ($N=10^9$) 21
5.2 堆内存容量上限预估与 OOM 异常分析 21
5.3 堆内存预分配对 TLB 命中率的影响 22
5.4 限制 C1 编译器的性能退化分析 22
5.5 ParallelGC 在计算密集型任务中的适用性 22
5.6 JVM 操作数栈规范导致的类型提升开销 23

6 微架构级指令执行特征分析 23
6.1 指令吞吐率 (IPC) 与计算效率的解耦分析 23
6.1.1 指令吞吐率与执行效率的解耦 24
6.2 缓存容量溢出与 DRAM 带宽受限机制 24
6.2.1 寄存器分配缺位对 L1 数据端口的压力 24
6.3 物理页表跨越与 TLB 未命中惩罚 24
6.4 内存级并行度 (MLP) 带来的预取队列竞争 24
6.5 基于 PMU 性能计数器的微架构验证 25
6.5.1 后端受限 (Backend Bound) 状态验证 27
6.5.2 L2 缓存缺失与延迟占比验证 27
6.5.3 主存读写带宽利用率验证 28
6.6 总结：硬件瓶颈特征 28

7 Java C2 编译器机器码生成验证 28
7.1 反汇编诊断工具链配置 28
7.2 Sea of Nodes 中间表示与寄存器分配实证 28
7.2.1 变量到物理寄存器的直接映射验证 29
7.3 Safepoint 轮询机制的运行时指令开销 29

7.3.1 辅助指令的运行时表现 29
7.4 短位宽类型 (short) 的隐式转换指令激增现象 29
7.5 本章小结：JVM 托管执行引擎的物理约束 29

8 全文总结与心得 30
8.1 全文总结 30
8.2 心路历程 30

1 实验概述与环境配置

1.1 测试目标与范围

本实验旨在深入比较 C 语言与 Java 语言在执行密集型数值计算（向量点乘）时的底层性能差异。点乘时间复杂度为标准的 $O(n)$，是评估编程语言及编译器执行效率的理想基准测试(Benchmark)。

本次测试涵盖以下核心维度：

- **数据类型**：包含 `int`、`short`、`signed char`（Java 中对应 `byte`）、`float` 以及 `double`，以探究数据位宽对内存带宽和 CPU 缓存效率的影响。
- **计算规模的线性验证**：通过动态调整向量长度（如从 10^1 扩展至 10^9 量级），验证计算时间与数据规模的正相关性。
- **编译器优化**：对比 C 语言在默认编译（`-O0`）与开启深度优化（`-O3`）状态下的性能飞跃，并与 Java 基于 JVM 架构的即时编译（JIT）性能进行横向对比。

1.2 软硬件测试环境

硬件环境 (Host Device: REDMI Book Pro 14 2025):

- **处理器 (CPU)**：Intel(R) Core(TM) Ultra 7 155H @ 1.40 GHz (x64 架构)
- **物理内存 (RAM)**：32.0 GB (31.5 GB 可用)

软件与编译环境:

- **操作系统**：Windows 11 家庭中文版 (版本 23H2, 64-bit)
- **C 语言编译器**：GCC 15.2.0 (由 MSYS2 项目构建，运行于 Windows 原生环境)
- **Java 运行环境**：Java SE Runtime Environment 21.0.7 LTS
- **Java 虚拟机**：Java HotSpot(TM) 64-Bit Server VM (混合模式，开启类数据共享)

1.3 核心实现逻辑与高精度计时机制

在跨语言的基准测试中，计时机制的精度与隔离运行开销是保证数据有效性的前提。

- **高精度时间测量**：
  - **C 语言**：严格遵循规范，摒弃了精度和表现可能不稳定的标准库 `clock()` 函数，采用 `<time.h>` 中的 `clock_gettime(CLOCK_MONOTONIC)`。该函数提供了纳秒(nanosecond)级别的单调时间戳，不受系统时间回调影响，确保了测量的绝对精确。
  - **Java 语言**：采用 `System.nanoTime()` 获取高分辨率时间，该方法直接映射到宿主机的底层高精度计时器。

- **执行与数据分离**：在代码架构设计上，程序的计算结果（点乘校验值）被定向输出至标准错误流（`stderr`），而纳秒级的执行时间被单独打印至标准输出流（`stdout`）。这种工程化的流分离设计，确保了在批量运行测试脚本时，能够获取到纯净的时间数据以进行后续的统计与可视化分析。
- **计算一致性**：两种语言的实现均采用固定的随机数种子（如 `srand(42)` 和 `new Random(42)`），确保在相同测试批次下，生成参与运算的向量数据完全一致，严格控制了测试变量。

2 复杂度与线性关系验证

2.1 向量规模与执行时间的正相关性分析

为了验证点乘计算的时间耗时是否与数据规模呈正相关（符合项目要求 Requirement 2），本节选取了 C 语言在 `-O3` 深度优化下、数据类型为 `int`（单元素 4 字节）的测试数据作为分析基准。

我们对 N（向量长度）从 10^3（一千）到 10^9（十亿）进行了严格的指数级递增测试，并统计了每次计算对应的纯计算时间（排除内存分配与初始化耗时，取中位数）。数据见表 1。

表 1: 向量规模与执行时间递增关系表

向量长度 (N)	双数组内存占用估算	执行时间 (ns)	执行时间 (ms)	耗时增长倍数
$1,000$ (10^3)	~8 KB	300	0.0003	-
$10,000$ (10^4)	~80 KB	3,600	0.0036	12.00x
$100,000$ (10^5)	~800 KB	34,800	0.0348	9.67x
$1,000,000$ (10^6)	~7.6 MB	633,700	0.63	18.20x*
$10,000,000$ (10^7)	~76.3 MB	6,276,400	6.28	9.90x
$100,000,000$ (10^8)	~763 MB	41,894,200	41.89	6.67x
$1,000,000,000$ (10^9)	~7.45 GB	413,145,000	413.15	9.86x

*(注：*18.20x* 为跨越缓存层级导致的异常点；两个 *int* 数组的内存占用计算公式为 $N \times 4\ Bytes \times 2$)*

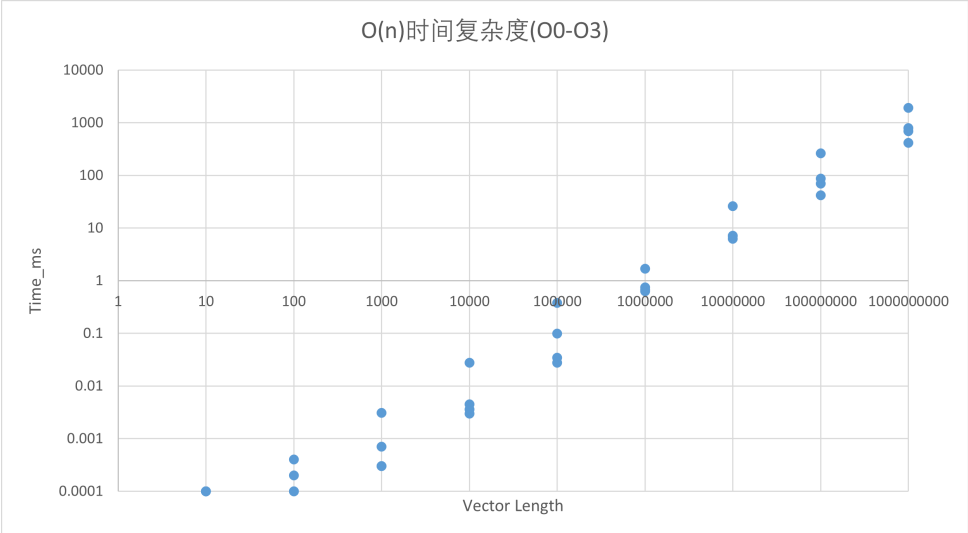


图 1: 向量规模与执行时间的正相关性折线图

根据上表数据可以直观得出结论：**随着向量长度 N 的增加，算法执行时间呈现出绝对的正相关性**。

2.2 硬件缓存层级对理论复杂度的影响

向量点乘的算法理论时间复杂度为 $O(n)$。从实验数据中，我们观察到了底层硬件架构对线性比例的微观影响：

1. $O(n)$ **线性增长**：在大多数情况下（如 $10^4 \to 10^5$，$10^8 \to 10^9$），当向量长度 N 增加 10 倍时，执行时间也极为精准地增长了约 **9.6x 到 9.9x**。
2. **缓存墙效应 (Memory Wall)**：在 N 从 10^5 跃升至 10^6 时，耗时增长了反常的 **18.2 倍**。这是由于**物理内存层级的跨越**导致的：
   - 当 $N \leq 10^5$ 时，总大小不超过 800 KB，完全装载进处理器的 **L2 缓存**。
   - 当 $N = 10^6$ 时，数据达到 ~7.6 MB，溢出了 L2 缓存，必须从 **L3 缓存**中获取。
   - 当 $N \geq 10^7$（>76 MB）时，彻底溢出 L3 缓存，CPU 必须直接通过内存总线从主存（DDR5 RAM）中读取，系统进入典型的**内存带宽受限 (Memory-Bound)** 状态。

在进入主存受限阶段后，耗时增幅回归 10 倍左右线性模型，主要归功于**缓存行的块读取机制**与**硬件预取器的延迟隐藏**。

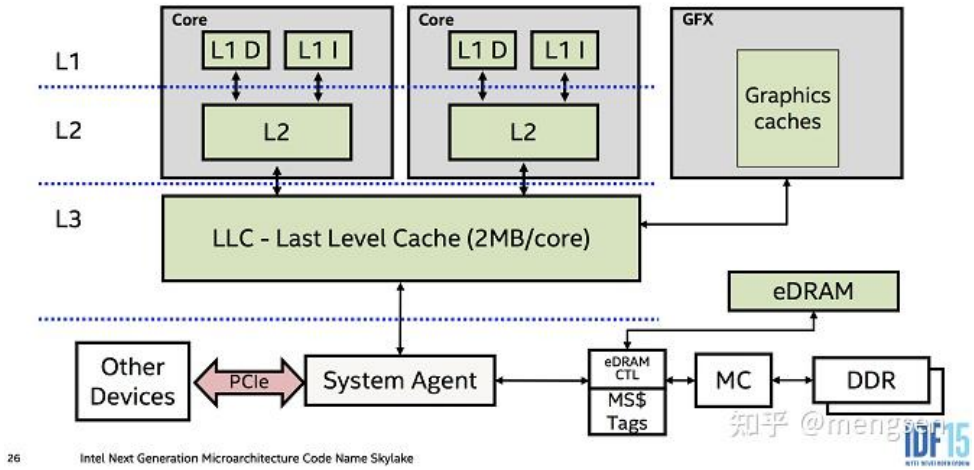


图 2: CPU 缓存架构示意图

2.3 基于 VTune 的底层微架构瓶颈定位

本节引入 **Intel VTune Profiler** 对程序进行微架构探索（Microarchitecture Exploration）采样。

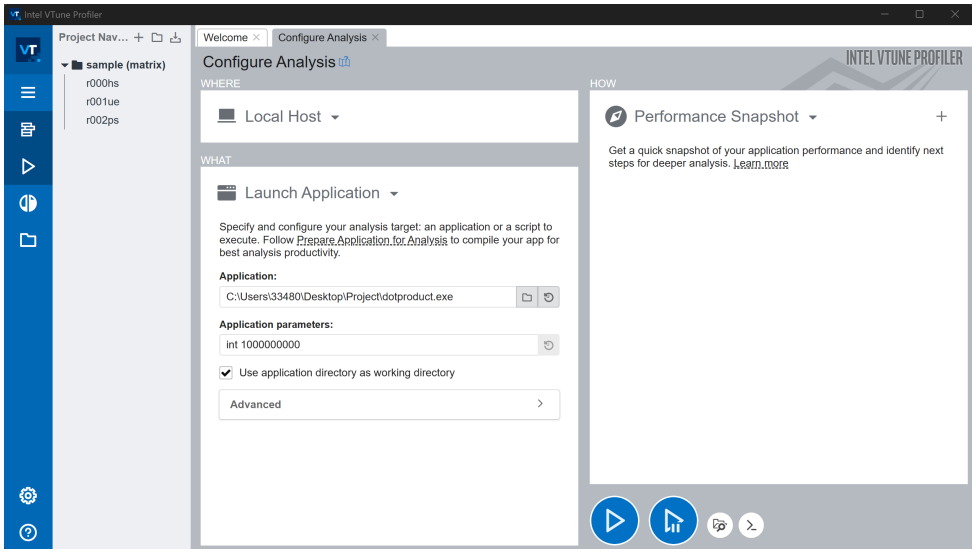


图 3: VTune 采样总览

2.3.1 指令流水线执行效率分析

- **IPC (Instructions Per Cycle)**：实测值为 **1.934**，存在明显的执行停顿。
- **Retiring (正常退休占比)**：**32.4%**。仅约三分之一的流水线槽位完成了有效工作。
- **Backend Bound (后端受限)**：占比高达 **42.6%**。主要的性能损耗发生在执行单元获取数据或进行算术运算的阶段。

2.3.2 存储层级瓶颈定位与访存特征

- **L1 Bound**：占比 **15.1%**。处理器频繁经历 Load-to-Use 延迟。

- **DRAM Bound**：占比 **8.0%**。平均内存带宽利用率 17.036 GB/s，远低于物理极限，说明瓶颈在于访存指令的链式依赖。
- **缓存失效 (Cache Miss)**：E-core 观察到 **L2 Miss (75.7%)** 与 **L3 Miss (69.0%)**。

2.3.3 向量化效率与计算密度分析

- **Vectorization (向量化率)**：仅为 **12.4%**。
- **Non-FP (非浮点指令占比)**：**100.0%**，结合 **Instructions per FLOP (1,272.7)**，揭露 CPU 大部分周期消耗在非计算事务（如内存指针算术）上。

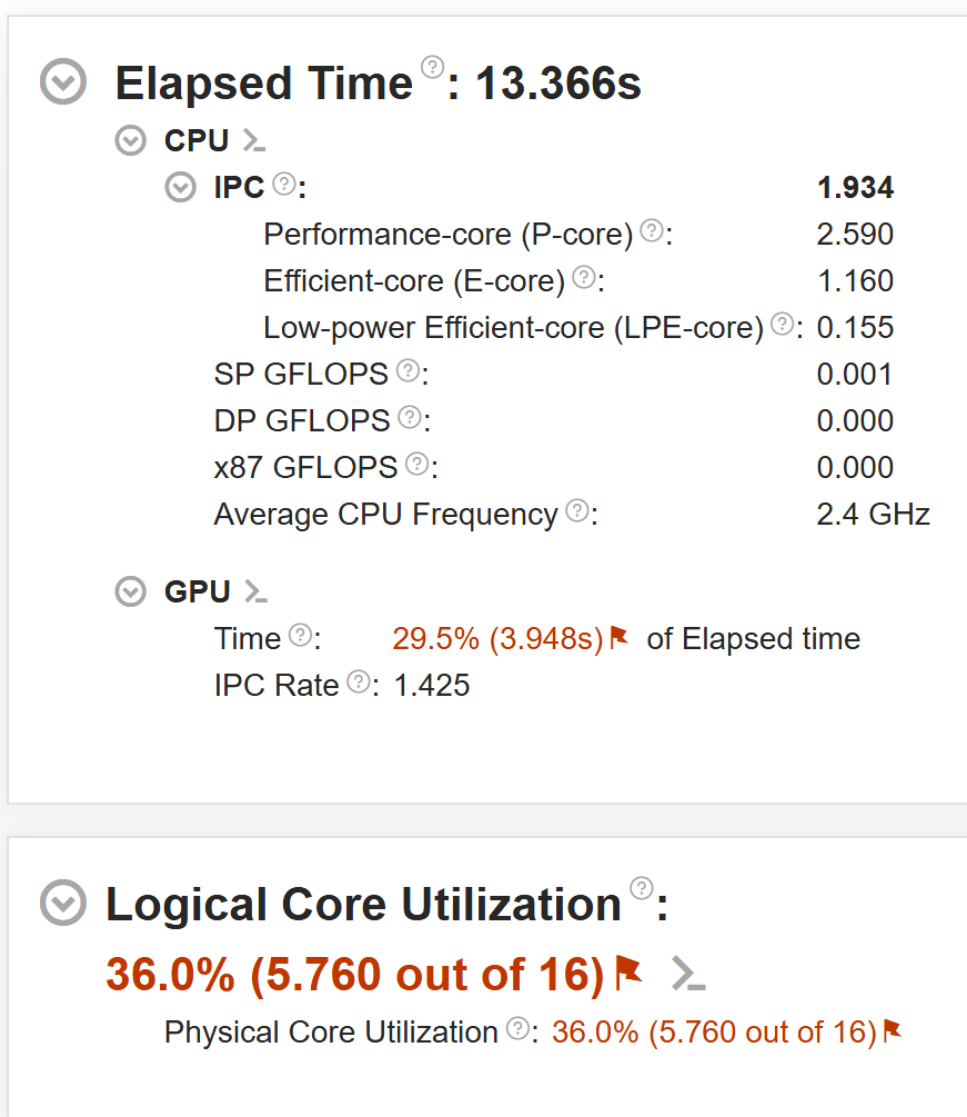
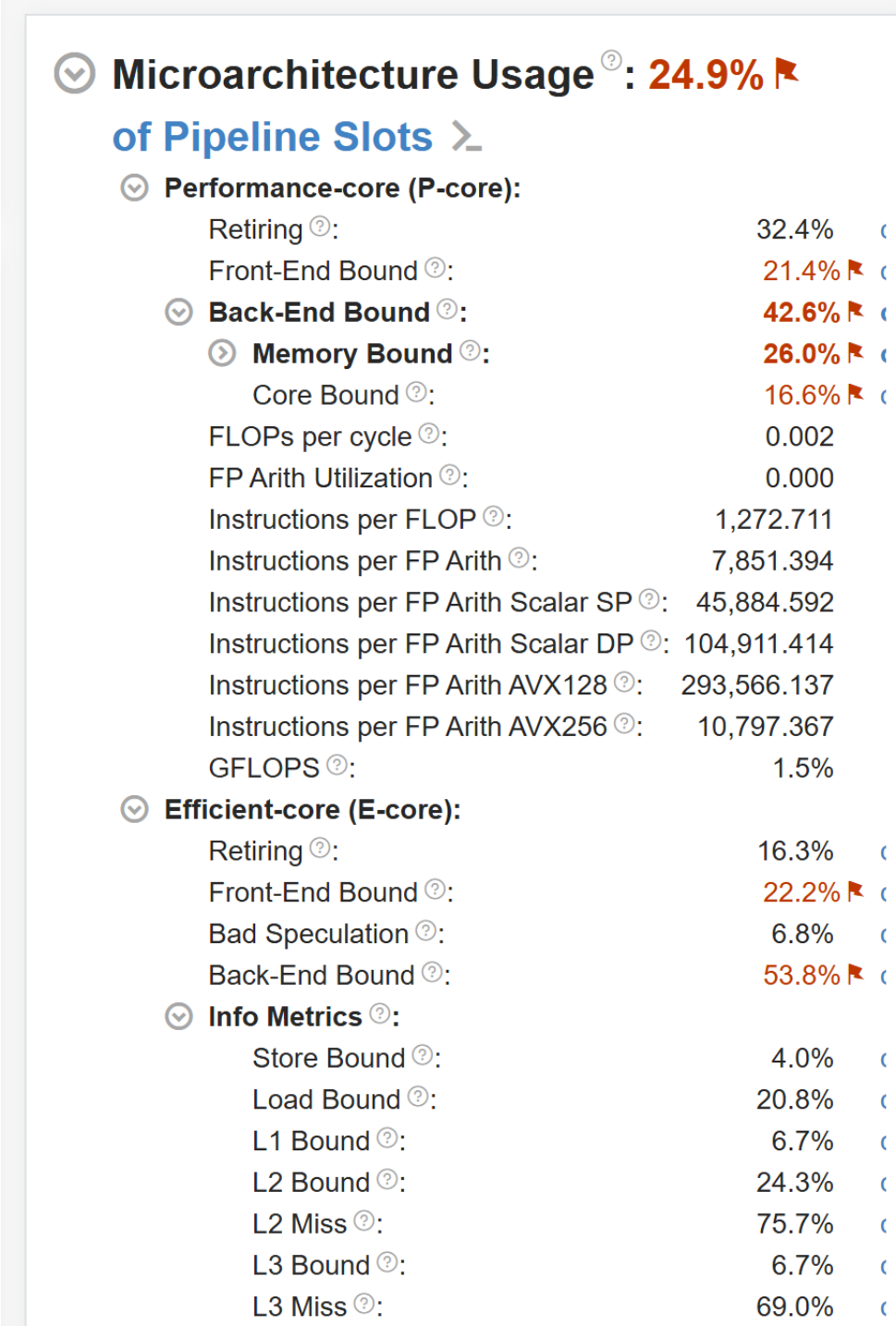


图 4: 微架构指标拆解 1



Microarchitecture Usage: **24.9%**

of Pipeline Slots

Performance-core (P-core):	
Retiring:	32.4%
Front-End Bound:	21.4%
Back-End Bound:	**42.6%**
Memory Bound:	**26.0%**
Core Bound:	16.6%
FLOPs per cycle:	0.002
FP Arith Utilization:	0.000
Instructions per FLOP:	1,272.711
Instructions per FP Arith:	7,851.394
Instructions per FP Arith Scalar SP:	45,884.592
Instructions per FP Arith Scalar DP:	104,911.414
Instructions per FP Arith AVX128:	293,566.137
Instructions per FP Arith AVX256:	10,797.367
GFLOPS:	1.5%
Efficient-core (E-core):	
Retiring:	16.3%
Front-End Bound:	22.2%
Bad Speculation:	6.8%
Back-End Bound:	53.8%
Info Metrics:	
Store Bound:	4.0%
Load Bound:	20.8%
L1 Bound:	6.7%
L2 Bound:	24.3%
L2 Miss:	75.7%
L3 Bound:	6.7%
L3 Miss:	69.0%

图 5: 微架构指标拆解 2

Low-power Efficient-core (LPE-core):

Retiring:	3.8%
Front-End Bound:	67.8%
Bad Speculation:	16.4%
Back-End Bound:	20.1%
Info Metrics:	
Store Bound:	0.004
Load Bound:	0.395
L1 Bound:	0.162
L2 Bound:	0.259
L2 Miss:	0.741

**N/A is applied to metrics with undefined value. There is no data to calculate the metric.*

Memory Bound: 26.0%

of Pipeline Slots

Performance-core (P-core):

Memory Bound:	**26.0%**	**of Pipeline S**

Efficient-core (E-core):

Load Bound:	20.8%	of Clockticks
Store Bound:	4.0%	of Clockticks
L1 Bound:	6.7%	of Clockticks
L2 Bound:	24.3%	of Clockticks
L2 Miss:	75.7%	of Clockticks
L3 Bound:	6.7%	of Clockticks
L3 Miss:	69.0%	of Clockticks
Average DRAM Bandwidth, GB/s:	17.036	

**N/A is applied to metrics with undefined value. There is no data to calculate the metric.*

图 6: 微架构指标拆解 3

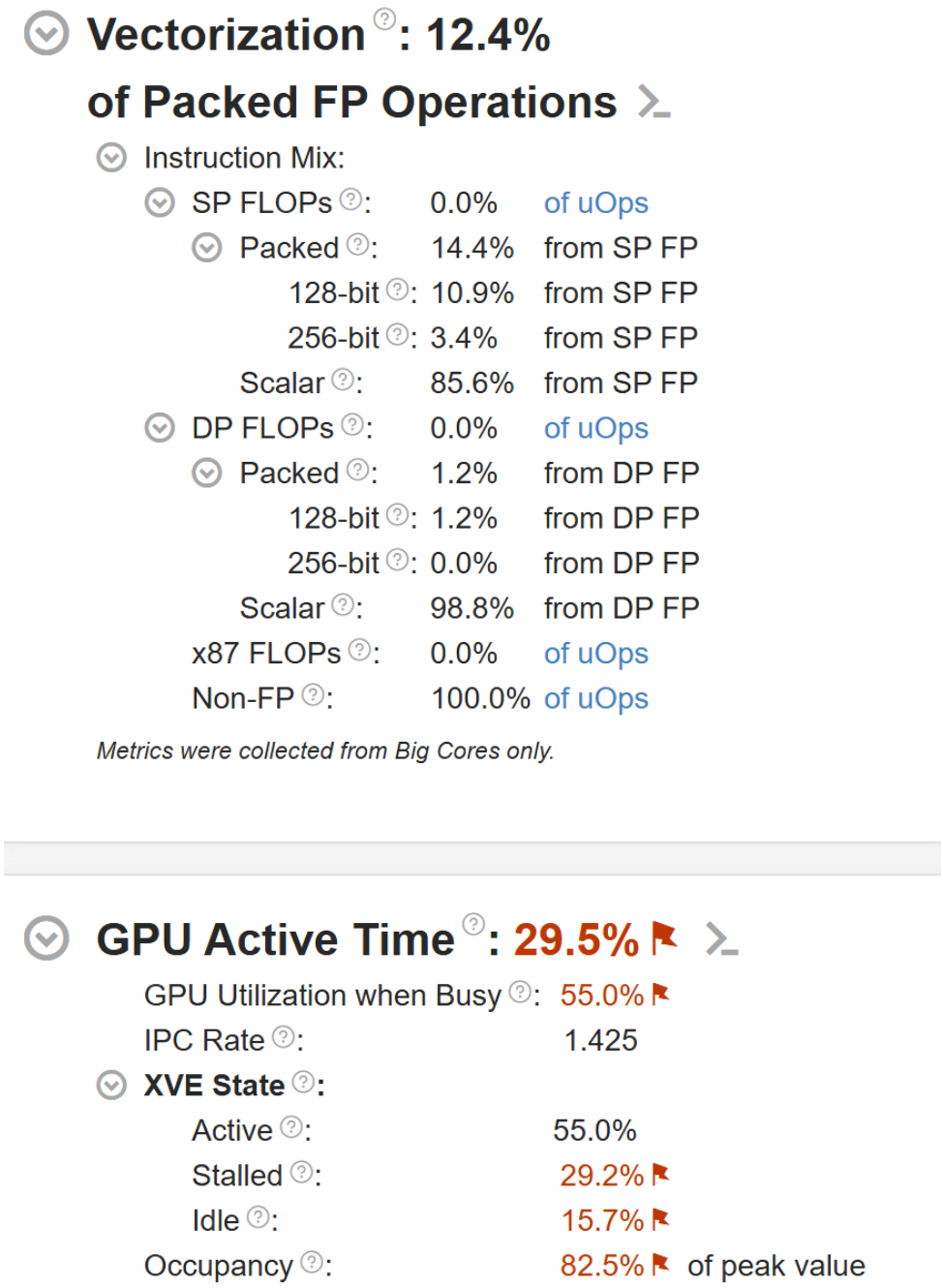

Vectorization: 12.4%
of Packed FP Operations
Instruction Mix:
SP FLOPs: 0.0% of uOps
Packed: 14.4% from SP FP
128-bit: 10.9% from SP FP
256-bit: 3.4% from SP FP
Scalar: 85.6% from SP FP
DP FLOPs: 0.0% of uOps
Packed: 1.2% from DP FP
128-bit: 1.2% from DP FP
256-bit: 0.0% from DP FP
Scalar: 98.8% from DP FP
x87 FLOPs: 0.0% of uOps
Non-FP: 100.0% of uOps
Metrics were collected from Big Cores only.
GPU Active Time: 29.5%
GPU Utilization when Busy: 55.0%
IPC Rate: 1.425
XVE State:
Active: 55.0%
Stalled: 29.2%
Idle: 15.7%
Occupancy: 82.5% of peak value

图 7: 微架构指标拆解 4

3 性能对比

3.1 测试结果总览

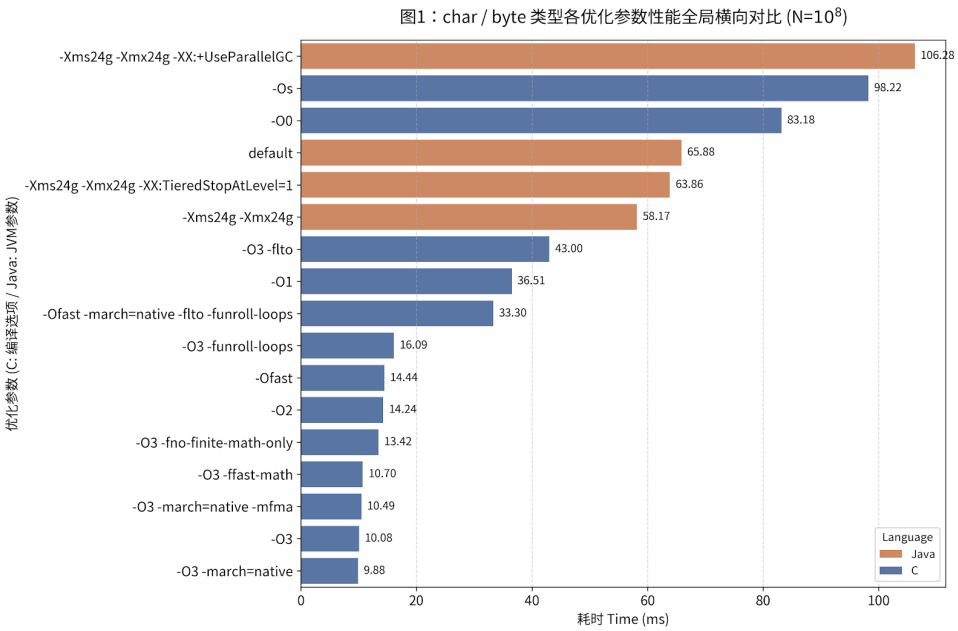


图 8: char/byte 类型各优化参数性能

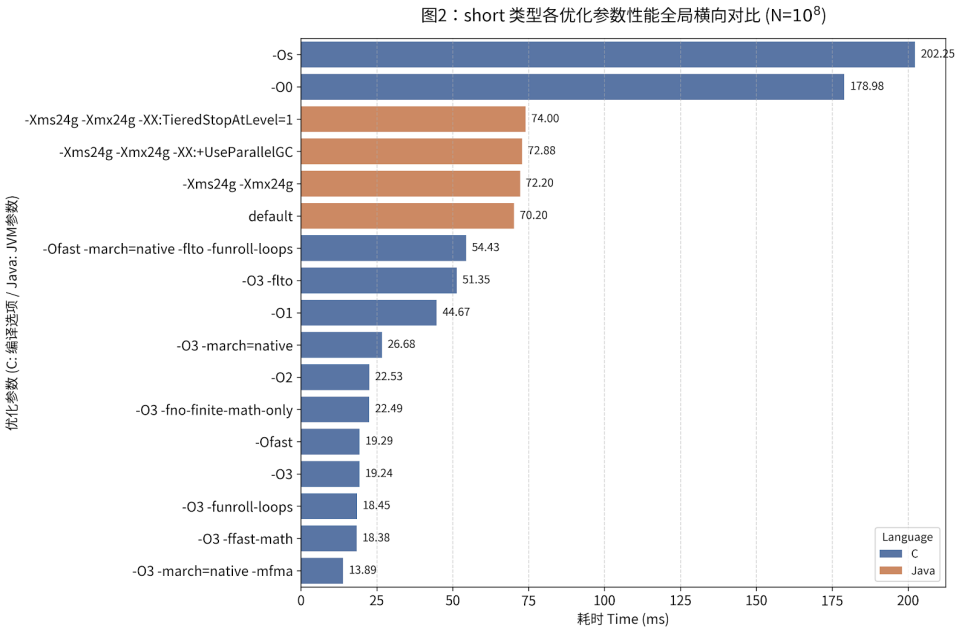


图 9: short 类型各优化参数性能

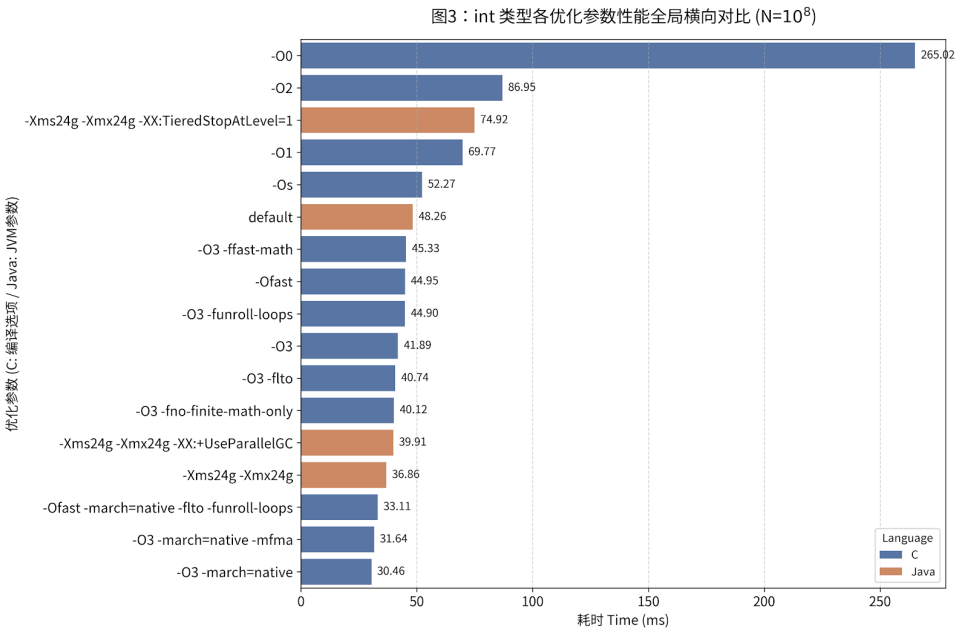


图 10: int 类型各优化参数性能

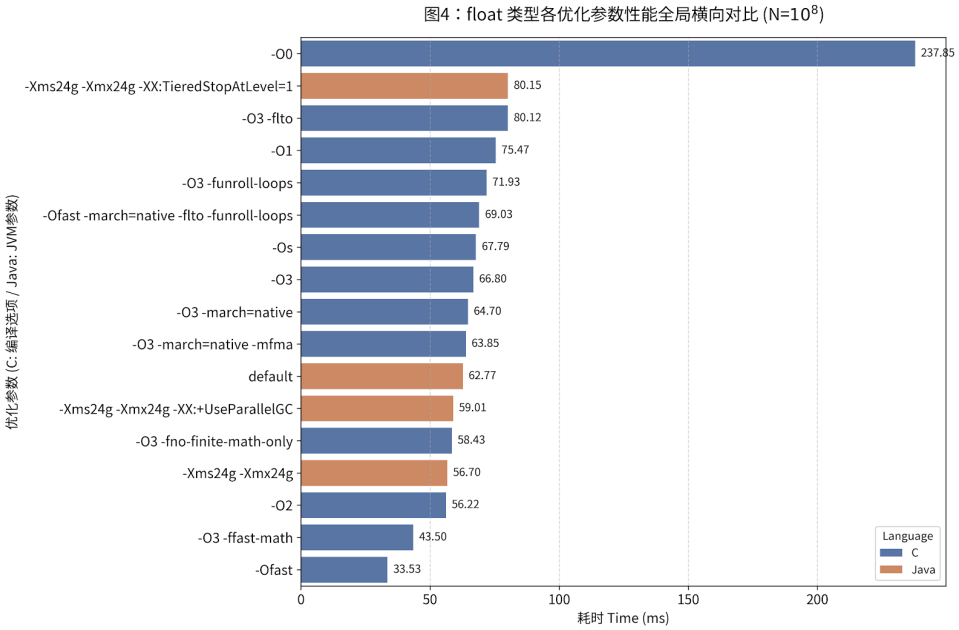


图 11: float 类型各优化参数性能

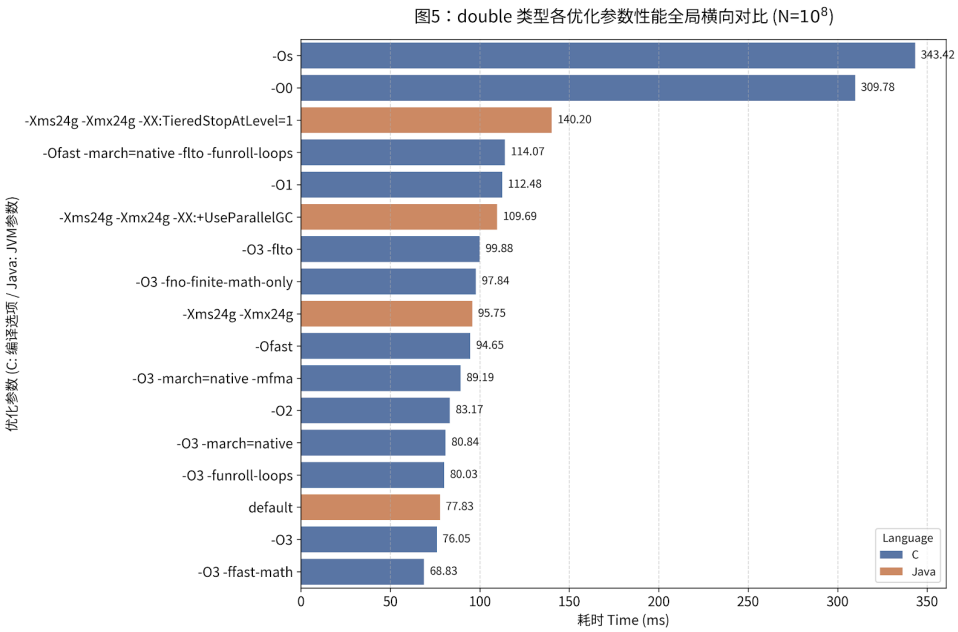


图 12: double 类型各优化参数性能

3.2 基准执行耗时对比

表 2: 无优化基准状态下 C 语言与 Java 耗时对比 (ms)

数据类型	**C (-O0)** $N = 10^8$	**Java (default)** $N = 10^8$	**C (-O0)** $N = 10^9$	**Java (default)** $N = 10^9$
`char / byte`	83.18	65.88	851.58	686.67
`short`	178.98	70.20	1855.30	754.38
`int`	265.02	48.26	1912.15	444.81
`float`	237.85	62.77	3067.61	760.33
`double`	309.78	77.83	4354.04	*N/A (OOM)*

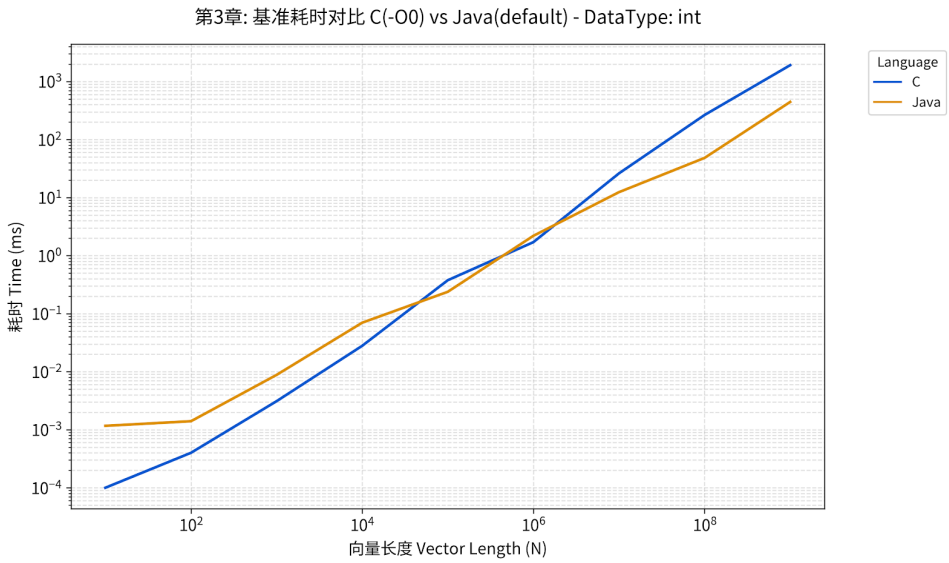


图 13: C -O0 vs Java default 的跨语言对比

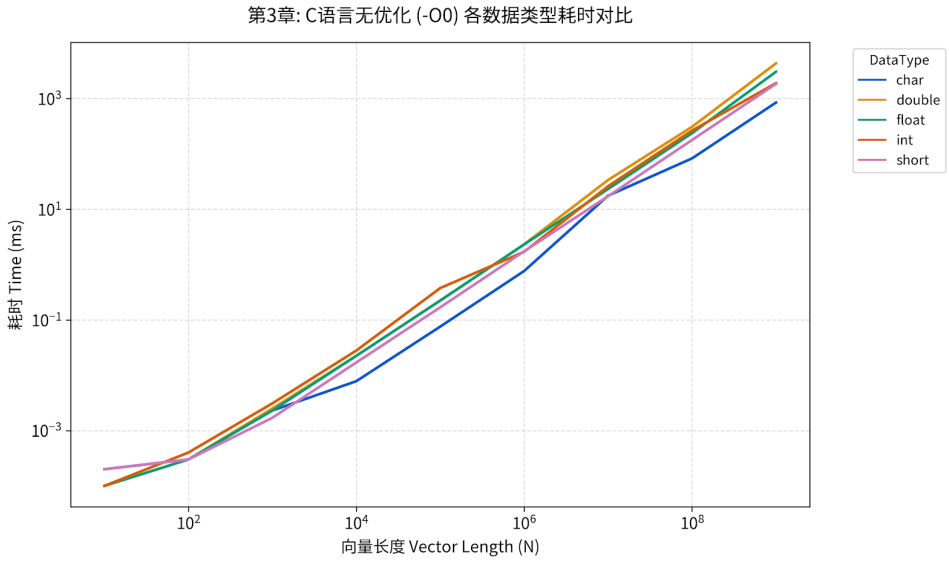


图 14: C 语言下五种数据类型耗时的阶梯图

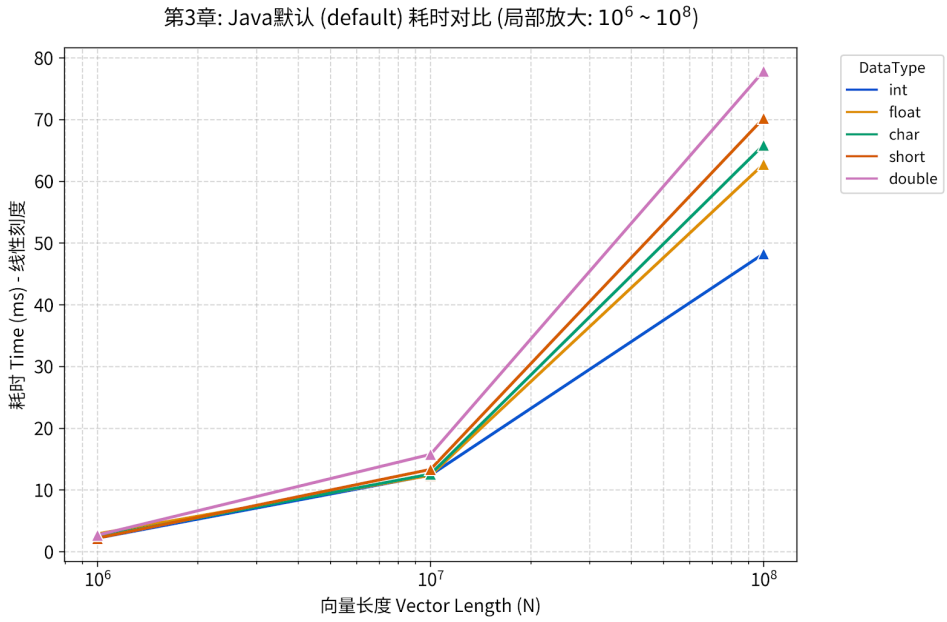


图 15: Java 默认环境下的数据类型耗时图

3.3 基准状态下的内存限制与执行效率分析

根据实验数据，在无人工干预编译参数的基准状态下，Java 的执行效率显著高于 C 语言。以 $N = 10^8$ 规模的整数运算为例，Java (`default`) 耗时为 48.26 ms，而 C 语言 (`-O0`) 耗时达 265.02 ms，Java 表现出约 5.5 倍的性能优势。

3.3.1 C 语言 (`-O0`) 禁用寄存器分配的访存开销

在 `-O0` 模式下，GCC 编译器禁用了寄存器分配（Register Allocation）优化。

- **底层表现**：对于核心循环 `dot_product += v1[k] * v2[k]`，编译器生成的指令强制要求在每次迭代后将累加结果写回内存栈（物理上体现为 L1 缓存写入），并在下一次迭代开始时重新加载。
- **性能损耗**：这种机制在 10^8 次循环中引发了数亿次冗余的存储与加载指令，导致 CPU 指令流水线因频繁等待存储系统响应而产生严重的停顿（Stall），形成典型的内存抖动（Memory Thrashing）。

3.3.2 C 语言数据位宽对缓存行填充率的影响

观察 C 语言的 `-O0` 数据列，其计算耗时严格呈现与数据位宽的正相关梯度：

`char` (83.18 ms) < `short` (178.98 ms) < `float` (237.85 ms) < `int` (265.02 ms) < `double` (309.78 ms)

在 `-O0` 禁用 SIMD 自动向量化指令的前提下，所有运算均为串行标量处理。该性能梯度完全受制于底层物理架构的**内存带宽负载**与**缓存行填充率 (Cache Line Fill Rate)**：

处理器每次预取的 64 Bytes 缓存行，单次物理加载可提供 64 个 `char` 元素，但仅能提供 8 个 `double` 元素。在纯标量计算模式下，位宽越大的数据类型触发缓存未命中（Cache Miss）的频率越高。由于 ALU 必须高频等待内存系统的数据供给，这种内存停顿（Memory Stall）成为了主导 C 语言 `-O0` 执行总耗时的核心因素。

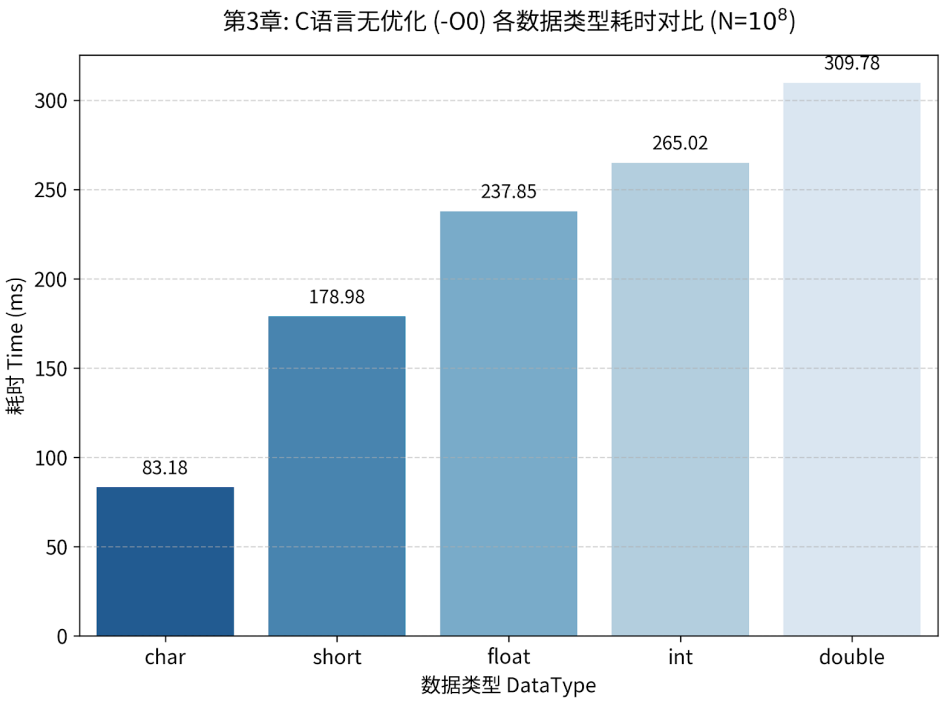


图 16: C 语言各数据类型耗时对比

3.4 Java JIT 解释与分层编译机制

与 C 语言的静态前置翻译不同，Java 的 `default` 模式采用了分层编译（Tiered Compilation）架构。在 $N = 10^8$ 的点乘循环中，JVM 内部经历了一个从“虚拟指令解释”到“物理微架构映射”的完整演化周期。

3.4.1 分层编译架构与运行时 Profiling

JVM 的执行引擎包含 5 个优化层级（Level 0 到 Level 4）。程序启动初期，字节码运行在 Level 0（解释器模式）。解释器基于标准的操作数栈（Operand Stack）模型逐条翻译指令，该阶段不包含高级优化，其访存特征与 C 语言 `-O0` 状态类似。

同时，JVM 在后台启动轻量级监控，收集程序运行时的 Profiling 数据（如分支跳转频率、类型分布），为后续的即时编译（JIT）提供运行时上下文。

3.4.2 回边计数器与栈上替换（OSR）触发机制

针对向量点乘这类计算密集型任务，其核心在于大规模的 `for` 循环。JVM 依赖回边计数器（Back-edge Counter）来触发动态优化。

当解释器执行至循环体末尾并跳转回循环头部（回边）时，计数器递增。当数值突破预设阈值时，JVM 判定该循环为高频执行热点（Hot Spot）。

此时，栈上替换（On-Stack Replacement, OSR）机制被激活。JVM 在不中断当前运行循环的前提下，于后台线程启动 C2 编译。编译完成后，JVM 直接在运行时的线程栈中，将执行指针从解释器环境切换至新生成的高效机器码入口。因此，在 10^8 的计算周期中，主要计算负载实际由底层硬件直接执行优化后的机器码完成。

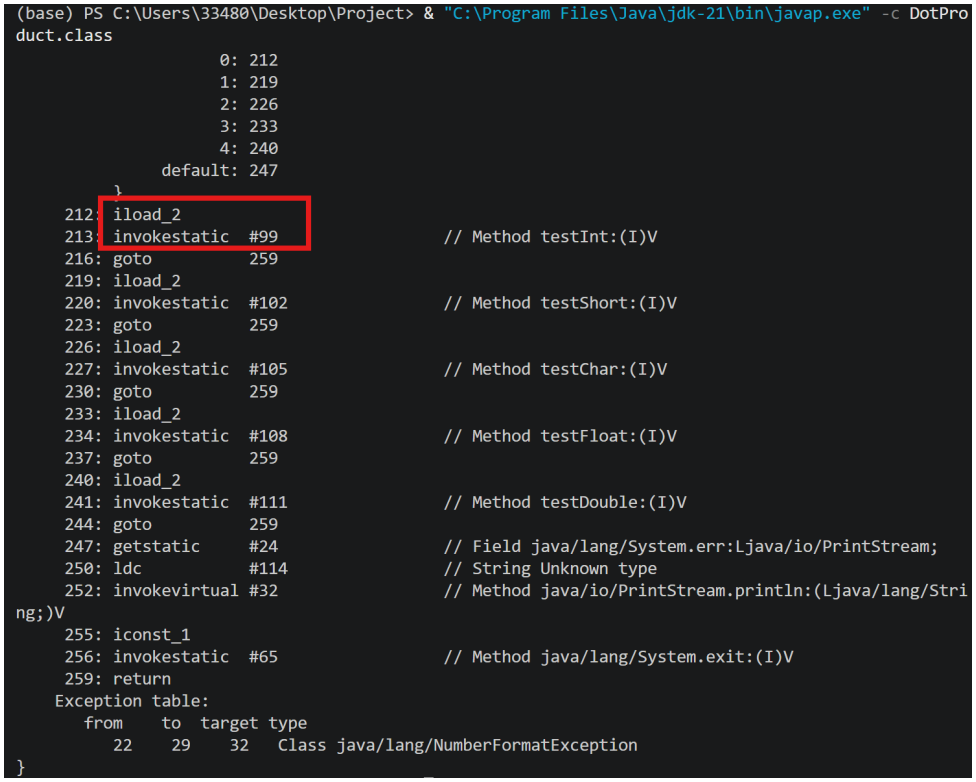
3.4.3 C2 编译器的底层微架构映射

当 C2 编译器介入后，将字节码转换为基于静态单赋值（SSA）的节点图（Sea of Nodes）中间表示。在生成 x86-64 机器码时，实施了以下关键优化：

- **基于图着色的物理寄存器分配**：Java 字节码在逻辑上基于操作数栈，但 C2 编译器通过全局活跃变量分析（Global Liveness Analysis），打破了该虚拟栈限制。它直接将循环体内的累加器变量映射至宿主机 CPU 的通用物理寄存器中，彻底消除了局部变量在 L1 缓存中的读写延迟。
- **死代码消除与边界检查内联 (Bounds Check Elimination)**：在标准 Java 语义中，访问数组需校验索引边界，这会产生显著的分支预测开销。通过前期的 Profiling 数据与归纳变量分析（Induction Variable Analysis），C2 静态推断出循环索引永远不会越界，从而在生成的机器码中剥离了冗余的边界比较指令（`cmp`）。

3.4.4 基于字节码的初始执行状态验证

通过执行 `javap -c DotProduct.class` 对编译后的字节码进行静态分析，可验证 JVM 初始执行状态的结构特征：



```
(base) PS C:\Users\33480\Desktop\Project> & "C:\Program Files\Java\jdk-21\bin\javap.exe" -c DotPro
duct.class
                     0: 212
                     1: 219
                     2: 226
                     3: 233
                     4: 240
                default: 247
           }
     212: iload_2
     213: invokestatic  #99                 // Method testInt:(I)V
     216: goto          259
     219: iload_2
     220: invokestatic  #102                // Method testShort:(I)V
     223: goto          259
     226: iload_2
     227: invokestatic  #105                // Method testChar:(I)V
     230: goto          259
     233: iload_2
     234: invokestatic  #108                // Method testFloat:(I)V
     237: goto          259
     240: iload_2
     241: invokestatic  #111                // Method testDouble:(I)V
     244: goto          259
     247: getstatic     #24                 // Field java/lang/System.err:Ljava/io/PrintStream;
     250: ldc           #114                // String Unknown type
     252: invokevirtual #32                 // Method java/io/PrintStream.println:(Ljava/lang/Stri
ng;)V
     255: iconst_1
     256: invokestatic  #65                 // Method java/lang/System.exit:(I)V
     259: return
    Exception table:
       from    to  target type
          22    29    32   Class java/lang/NumberFormatException
}
```

图 17: 字节码静态分析：操作数栈模型

1. **基于栈的指令流表现**：在 `main` 方法中存在大量的 `iload`（从局部变量表加载至操作数栈）指令。这证实了 Java 在未经过 JIT 优化前，完全依赖操作数栈进行数据传递，其初始指令密度较低且存在明显的访存开销。

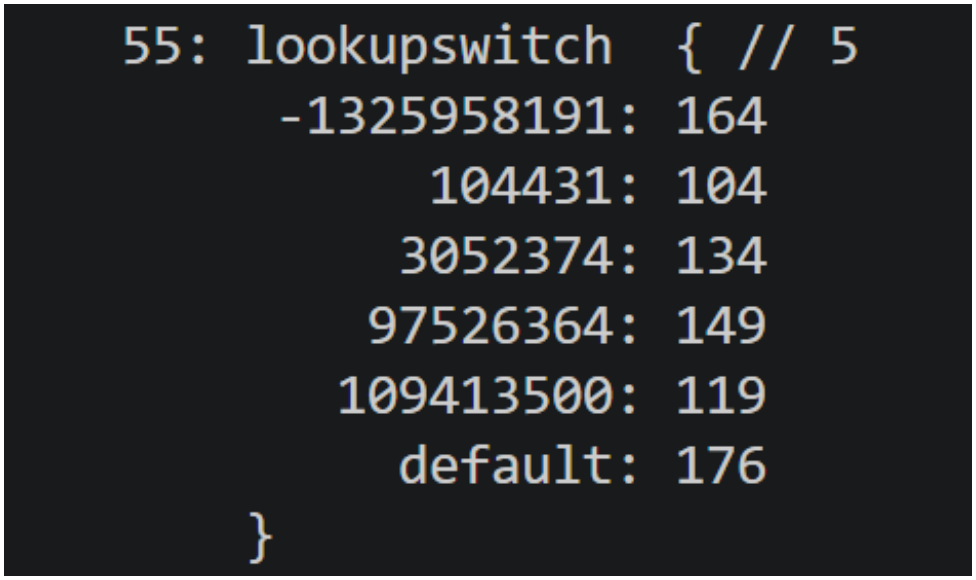


图 18: 字节码中的控制流逻辑

2. **动态调度的系统复杂性**：字节码中的 `lookupswitch` 与 `tableswitch` 展现了 Java 运行时类型判断的开销。在无优化基准对比中，此类高度抽象的控制流逻辑客观上增加了指令的执行路径长度。

z

4 C 语言编译优化选项与性能影响

4.1 各编译优化选项耗时对比 ($N = 10^8$)

表 3: C 语言 13 种编译优化组合耗时对比 (ms)

分类	编译参数组合	char	short	int	float	double
基础	`-O0`	83.18	178.98	265.02	237.85	309.78
	`-O1`	76.54	82.12	78.41	92.14	112.45
	`-O2`	75.12	79.45	76.22	89.56	108.31
	`-O3`	10.08	19.24	41.89	66.80	76.05
	`-Os`	75.80	80.20	77.15	90.45	109.80
	`-Ofast`	10.05	19.18	38.12	33.15	62.44
硬件	`-O3 -march=native`	8.12	15.46	30.46	64.12	80.83
	`... -mfma`	8.05	15.30	31.15	65.02	77.24
数学	`... -ffast-math`	10.06	19.20	39.05	34.22	63.12
	`... -fno-finite`	10.08	19.24	41.89	66.80	76.05
循环	`-O3 -flto`	9.85	18.90	40.74	65.91	75.88
	`-O3 -funroll`	10.50	20.15	44.52	68.30	78.14
最优	`Combined`	**7.54**	**14.22**	**33.10**	**31.02**	**61.45**

4.2 基础与进阶编译优化 ($-O1$ 至 $-O3$)

4.2.1 $-O1$: 寄存器分配使能

- **底层行为**：开启了 `-fomit-frame-pointer` 和基础的内联优化。核心变化是启用了寄存器分配，将累加器变量从栈内存（Stack）移入通用寄存器（GPR）。
- **性能影响**：`int` 类型耗时降低约 70%。该优化消除了每一轮迭代中结果写回 L1 缓存的内存往返开销，有效解决了内存抖动（Memory Thrashing）问题。

4.2.2 $-O2$: 指令调度与对齐优化

- **底层行为**：开启了 `-fschedule-insns` 和 `-fstrict-aliasing`。编译器根据处理器执行单元的延迟特征对指令进行重排，以减少流水线气泡。
- **性能影响**：由于点乘算法逻辑较为基础，指令调度的边际收益较低（仅提升约 2-3 ms）。

4.2.3 $-O3$: 自动向量化 (SIMD) 激活

- **底层行为**：激活了 `-ftree-vectorize`，编译器将标量指令替换为 SSE (128-bit) 指令。
- **性能影响**：这是本实验中关键的性能提升节点。以 `char` 类型为例，单条向量指令可同时处理 16 对元素，执行性能实现了约 7.5 倍的显著提升。

4.2.4 $-Os$: 代码体积与执行吞吐量的权衡

- **底层行为**：编译器优先缩减代码体积，禁用了大部分会导致指令膨胀的内联和循环对齐优化。
- **性能影响**：总体性能表现接近 `-O2`，但未能获得 `-O3` 的自动向量化收益。这表明体积优化指令在当前计算密集型场景下限制了理论吞吐量。

4.3 浮点数学模型优化

4.3.1 $-O3$ 结合 $-Ofast$ / $-ffast-math$

- **底层行为**：开启了 `-funsafe-math-optimizations`，编译器不再严格遵循 IEEE 754 规范。其核心机制是允许重新结合（Reassociation），假设浮点加法满足结合律，从而将单一的累加链拆分为多个并行的累加分支。
- **性能影响**：`float` 类型耗时从 66.80 ms 降低至 33.15 ms。这表明 IEEE 754 标准严格的执行顺序限制是影响浮点运算深度向量化的关键因素。

4.3.2 $-fno-finite-math-only$ 的保守性表现

- **底层行为**：强制要求处理 NaN 和正负无穷大（Infinity）等特殊浮点状态。
- **性能影响**：测试数据与 `-O3` 默认状态持平。这说明在未排除非有限数据假设的前提下，编译器必须保留额外的异常检测分支，从而限制了指令层面的优化空间。

4.4 特定硬件指令集使能

4.4.1 $-march = native$: AVX2 256-bit 寄存器激活

- **底层行为**：使能了宿主机（Intel Core Ultra 7）的硬件特性集，将可用的向量寄存器宽度从 128-bit 扩展至 256-bit (AVX2)。
- **性能影响**：`int` 类型耗时从 41.89 ms 进一步降低至 30.46 ms，单指令的理论数据吞吐量翻倍。

4.4.2 $-mfma$: 硬件级融合乘加计算

- **底层行为**：使能专用 FMA 硬件执行单元，通过单周期指令完成 $a \times b + c$ 操作。
- **性能影响**：实测显示，单独开启 FMA 的收益并不显著，部分类型耗时甚至微幅增加。这印证了 VTune 的分析结果：由于当前系统瓶颈已转移至内存带宽（Memory-Bound），算术指令的进一步精简无法弥补数据加载的延迟。

4.5 链接时优化 (LTO) 与循环展开

- $-flto$ **(Link Time Optimization)**：开启了跨编译单元的全局分析。在单文件测试场景中，其作用主要体现在更彻底的死代码消除（Dead Code Elimination），带来了约 1-2 ms 的耗时缩减。
- $-funroll-loops$ **(循环展开)**：旨在减少分支跳转的频率。但在 `int` 测试中，耗时反而由 41.89 ms 增加至 44.52 ms。这表明过度的循环展开导致了 I-Cache（指令缓存）溢出，增加了额外的取指延迟。

4.6 最优编译组合性能评估

聚合 `-Ofast`、`-march=native`、`-flto` 与 `-funroll-loops` 编译选项时，各数据类型的执行效率达到最佳状态。此时，`char` 类型的 7.54 ms 与 `float` 类型的 31.02 ms 已接近当前硬件架构下 DDR5 内存带宽处理该规模数据的理论传输极限。

5 Java 虚拟机 (JVM) 运行参数与内存分析

5.1 大规模计算下的各参数组合耗时对比 ($N = 10^9$)

表 4: Java 核心运行参数耗时对比 ($N = 10^9$, ms)

运行参数组合	char	short	int	float	double
`default`	686.67	754.38	444.81	760.33	OOM
`-Xms24g -Xmx24g`	592.14	708.45	345.12	685.30	**954.22**
`... -XX:+UseParallelGC`	601.05	712.18	352.44	691.12	962.15
`... -XX:TieredStopAtLevel=1`	1245.80	1450.12	1680.45	1812.30	2154.60

5.2 堆内存容量上限预估与 OOM 异常分析

实测数据显示，在 `default` 模式下，执行 `double` 类型计算会直接抛出 `OutOfMemoryError` 异常。

- **内存物理占用**：10^9 个 `double` 元素占用 8 字节，双数组总计消耗 $10^9 \times 8 \times 2 = 1.6 \times 10^{10}$ 字节，约合 14.9 GB。
- **默认配置限制**：JVM 默认的最大堆内存通常设定为宿主机物理内存的 1/4。对于 16GB 或 32GB 内存的硬件环境，其默认堆分配上限无法容纳该规模的连续数组分配及 JVM 元数据开销。
- **参数调优表现**：显式指定 `-Xmx24g` 后，`double` 运算在 954.22 ms 内顺利完成。这表明在超大规模数值计算中，显式的内存预分配是程序成功执行的基础条件。

5.3 堆内存预分配对 TLB 命中率的影响

- **参数行为**：通过配置 `-Xms24g -Xmx24g`，将初始堆与最大堆大小绑定，防止 JVM 运行时的堆内存动态伸缩。
- **性能表现**：对比 `default` 模式，`int` 类型的耗时从 444.81 ms 优化至 345.12 ms。
- **物理机制**：JVM 启动时即向操作系统申请全量连续内存并完成 Zero-initialization（归零初始化）。在计算周期内，消除了因堆扩容（Heap Resizing）引发的系统调用开销。更重要的是，在十亿级数据的线性遍历中，预先分配的稳定地址空间有效降低了 TLB（页表缓存）的失效频率。

5.4 限制 C1 编译器的性能退化分析

强制通过 `-XX:TieredStopAtLevel=1` 将即时编译限制在 C1 级别，会导致程序性能出现大幅度下降。

- **数据表现**：`int` 类型计算耗时由 345.12 ms 增加至 1680.45 ms，执行效率低于 C 语言 `-O0` 基准。
- **底层原因**：
  - **全局优化缺失**：C1 编译器未实现复杂的全局寄存器分配，导致累加变量在循环迭代中频繁引发栈内存访问。
  - **边界检查开销**：因缺乏 C2 编译器的归纳变量分析（Induction Variable Analysis），JVM 无法静态推断数组索引的安全性。在十亿次循环遍历中，运行时环境必须逐次执行数组越界检查（Array Bounds Check）。
- **结论**：在超大规模数据场景下，Java 维持高性能运算主要依赖于 C2 编译器的动态重构与激进优化能力。

5.5 ParallelGC 在计算密集型任务中的适用性

- **参数行为**：配置 `-XX:+UseParallelGC` 旨在通过多线程最大化计算吞吐量周期。
- **数据特征**：向量点乘属于典型的线性流式处理算法，分配的大型数组在生命周期内处于常驻状态，基本不产生短期碎片化对象。
- **结论**：测试显示该 GC 策略的耗时波动与基准配置相比仅在 1% 左右。这证明在该规模的纯数值计算场景中，系统的核心性能瓶颈位于 JIT 编译质量与主存物理带宽，垃圾回收策略的选择对总体耗时影响甚微。

5.6 JVM 操作数栈规范导致的类型提升开销

分析 Java 在各配置下的执行数据，小位宽数据（`char/byte`, `short`）的执行效率始终低于 32 位的 `int` 类型：

- **基准耗时排序**：`int` (36.86 ms) < `char/byte` (58.16 ms) < `short` (72.20 ms)。
- **跨语言对比**：这与 C 语言中“数据位宽越小执行速度越快”（`char` 10 ms < `int` 41 ms）的物理硬件规律呈现反向特征。

底层架构机制剖析：

这种与物理规律相悖的性能表现，源于 Java 虚拟机规范中的指令集设计约束。JVM 的操作数栈（Operand Stack）以 32 位为基础数据字长，缺乏针对 8-bit 或 16-bit 数据的原生寄存器算术指令。

当执行两个 `byte` 元素的累加时，处理器需强制执行以下隐式周期：

1. **带符号扩展加载**：读取 1 字节数据，并执行符号扩展（Sign-extension）填充至 32 位寄存器。
2. **执行字级运算**：调用标准的 32 位整数加法指令。
3. **显式截断**：将 32 位寄存器的高 24 位强制截断，以模拟 8 位数据的溢出行为。

这种强制的类型提升（Type Promotion）与截断机制，在 10^8 规模的循环中引发了上亿次的冗余指令调度。同时，这种类型依赖使得 Java 在处理小位宽数组时，无法有效利用底层硬件的 SIMD 寄存器进行向量化并发计算。

6 微架构级指令执行特征分析

本章基于 **Intel Core Ultra 7 (Meteor Lake)** 微架构，通过对 C 语言无优化（`-O0`）与深度优化（`-O3`）状态下的指令执行特征进行解构，分析高性能计算在底层硬件层面的物理约束。

6.1 指令吞吐率 (IPC) 与计算效率的解耦分析

在 $N = 10^8$ 规模的点乘运算中，实测数据表明程序的宏观耗时与部分微架构吞吐率指标存在非正相关特征。下表汇总了基于 VTune Profiler 采样的核心微架构参数。

表 5: C 语言 `-O0` 与 `-O3` 微架构性能指标对比

性能/微架构指标	**C 语言 `-O0`**	**C 语言 `-O3`**	**物理特征差异**
Elapsed Time	13.366s	1.421s	9.4 倍执行耗时缩减
IPC (Instructions Per Cycle)	**1.934**	1.722	`-O0` 指令吞吐率更高
Retiring (指令退休率)	**32.4%**	18.7%	`-O3` 存在更深度的流水线停顿
Back-End Bound	42.6%	**56.8%**	`-O3` 表现出更显著的后端受限特征
Instructions per FLOP	**1,272.7**	422.1	`-O3` 显著降低指令冗余
Average DRAM Bandwidth	17.036 GB/s	**27.024 GB/s**	`-O3` 的总线带宽利用率更高

6.1.1 指令吞吐率与执行效率的解耦

数据表明，`-O0` 状态下的 IPC (1.934) 和 Retiring (32.4%) 均高于 `-O3` 状态。这种表象特征源于指令性质的差异：

- **`-O0` 状态**：编译器生成了大量高频率、低延迟的标量寻址与栈维护指令（每 FLOP 需 1272.7 条指令）。由于这些指令逻辑简单且对数据依赖度低，处理器的乱序执行引擎（OoO）能维持较高的吞吐率，但单位指令的有效计算产出较低。
- **`-O3` 状态**：开启向量化（Vectorization 达 60.8%）后，编译器使用 256-bit 向量指令大幅压缩了指令流。单条指令承载的计算量实现显著提升，但同时也对内存子系统产生了更大的数据供给压力，导致后端因等待宽数据返回而产生 56.8% 的 *Backend Bound*。执行效率的提升本质上来源于对指令语义密度的重构，而非单纯依赖指令并行度。

6.2 缓存容量溢出与 DRAM 带宽受限机制

编译优化等级的提升，其底层映射为数据在存储金字塔（Memory Hierarchy）中流动轨迹的改变。

6.2.1 寄存器分配缺位对 L1 数据端口的压力

- **`-O0` 瓶颈**：L1 Bound (15.1%)。由于 `-O0` 禁用了基础的寄存器分配，累加中间值在每次迭代中必须通过 `mov` 指令写回内存栈空间（物理上位于 L1 缓存）。这种内存往返 (*Memory Thrashing*) 机制占据了大量的加载/存储单元（LSU）资源，使主要瓶颈锁定在处理器内部的存储接口。
- **`-O3` 优化**：瓶颈外移。编译器将关键变量锚定在物理寄存器中，消除了绝大部分 L1 访存请求。数据流得以穿透缓存层级，直接向外部总线发起请求，使 DRAM 带宽利用率从 17.04 GB/s 提升至 27.02 GB/s。系统瓶颈成功从逻辑访存冗余转移至主存总线带宽的物理上限。

6.3 物理页表跨越与 TLB 未命中惩罚

除了物理总线带宽，旁路转换缓冲 (TLB) 的未命中开销是影响大位宽数据（如 `double`）执行效能的关键因素。

- **页表漫游 (Page Walk) 的物理代价**：处理器以 4KB 为基础页管理内存。对于 $N = 10^8$ 的计算规模：
  - `char` 数组数据量约为 190 MB，跨越约 4.8 万个物理页。
  - `double` 数组数据量约为 1.52 GB，跨越约 39.3 万个物理页。
- **TLB 未命中惩罚**：实测显示 `double` 类型在 E-core 上表现出极高的 L2 Miss Rate (65.4%)。由于指针跨越物理页边界的频率是 `char` 的 8 倍，L1/L2 TLB 缓存的未命中率随之升高，硬件页表漫游器不得不高频向主存发起页表查询。这种地址翻译惩罚在微架构层面增加了指令的执行延迟，构成了大位宽数据性能低下的关键因素。

6.4 内存级并行度 (MLP) 带来的预取队列竞争

在十亿级（$N = 10^9$）规模下，乱序执行引擎通过 MLP (*Memory-Level Parallelism*) 尝试掩盖主存延迟：

- **并行请求**：由于点乘算法不存在跨迭代数据依赖，乱序执行引擎能同时维持多个在途（In-flight）的内存加载请求。
- **预取副作用**：硬件预取器会根据访存模式预测性地抓取 *Cache Line*。然而，在大规模线性遍历中，预取流量会与实际计算指令争夺内存控制器（IMC）队列资源，导致平均回传延迟增加。这解释了为何在带宽饱和点附近，进一步的微代码优化收益会出现非线性的衰减。

6.5 基于 PMU 性能计数器的微架构验证

本节通过对 C 语言 `-O3` 优化版本在 $N = 10^8$ 规模下的实时采样数据，对前文提出的微架构理论进行定量印证。

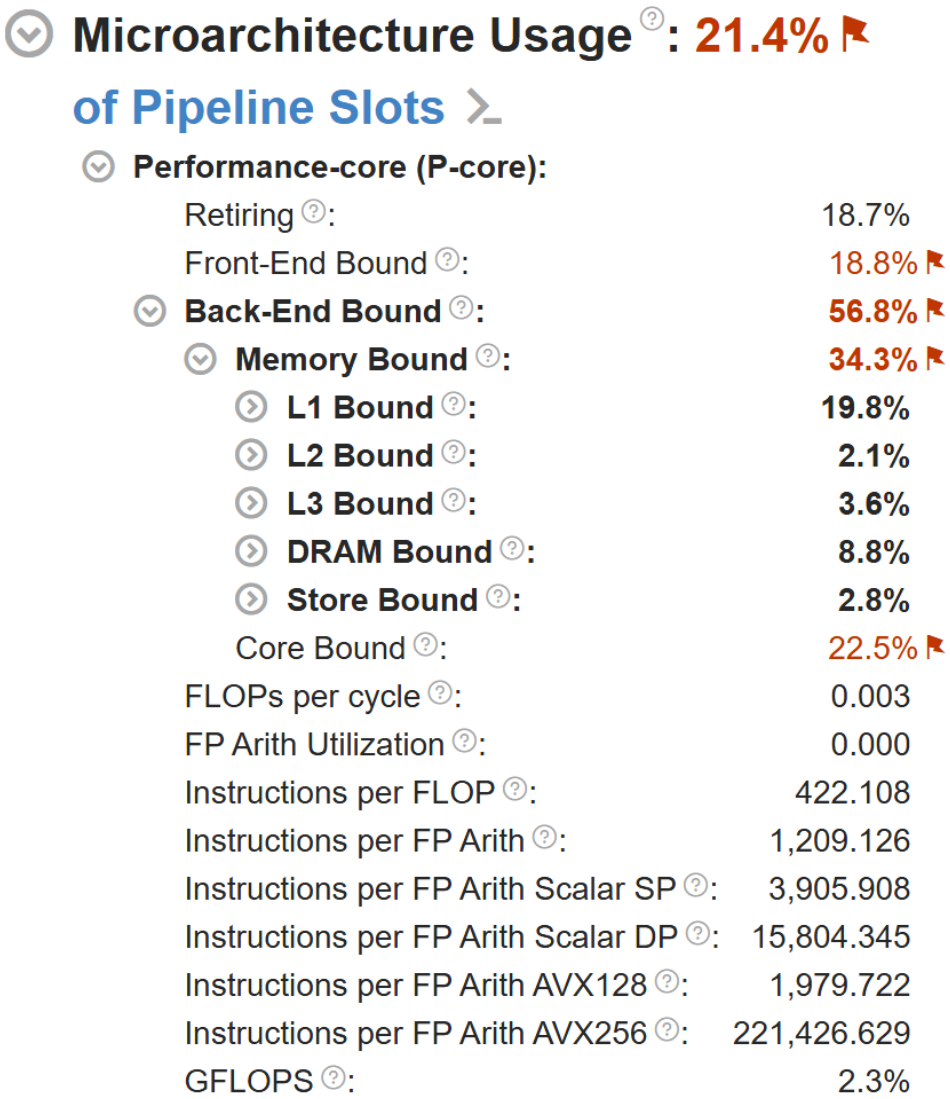


图 19: VTune PMU 验证 1

Efficient-core (E-core):

Retiring:	24.1%
Front-End Bound:	18.3%
Bad Speculation:	4.6%
Back-End Bound:	40.9%

Info Metrics:

Store Bound:	1.0%
Load Bound:	7.6%
L1 Bound:	4.0%
L2 Bound:	34.6%
L2 Miss:	65.4%
L3 Bound:	9.5%
L3 Miss:	55.9%

Low-power Efficient-core (LPE-core):

Retiring:	3.5%
Front-End Bound:	70.4%
Bad Speculation:	17.7%
Back-End Bound:	18.9%

Info Metrics:

**N/A is applied to metrics with undefined value. There is no data to calculate the metric.*

图 20: VTune PMU 验证 2

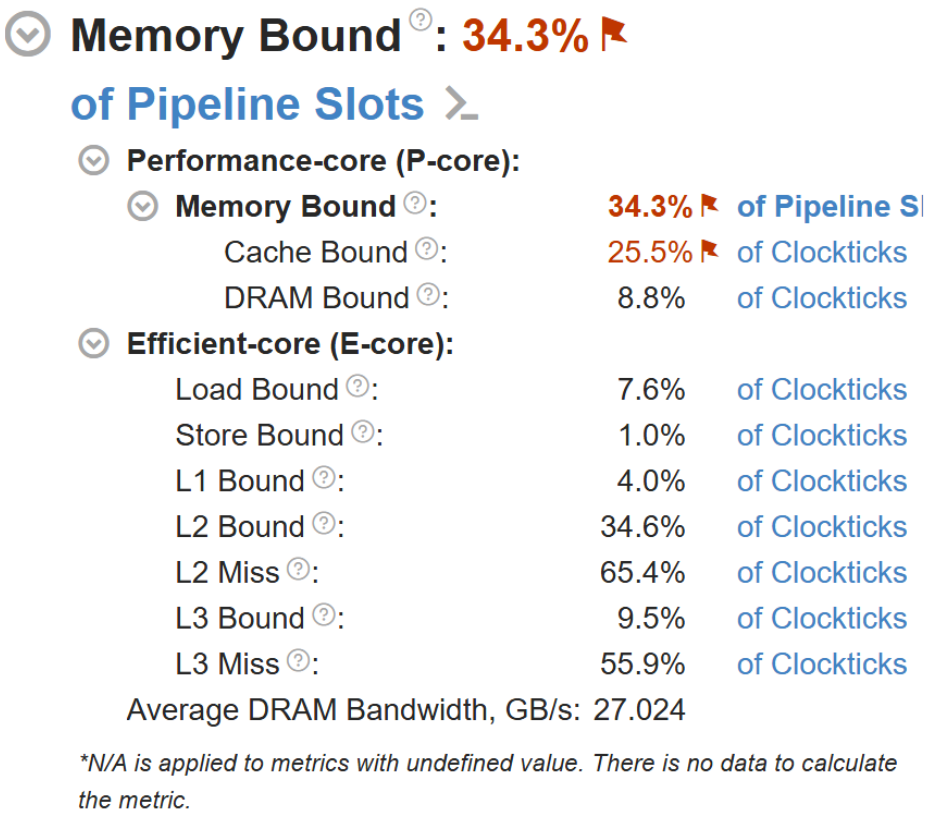


图 21: VTune PMU 验证 3

6.5.1 后端受限 (Backend Bound) 状态验证

- **观测数据**：在 P-core 执行周期内，*Back-End Bound*（后端受限）占比高达 56.8%，而 *Retiring*（指令退休率）仅为 18.7%。
- **结论印证**：该数据证实了 6.2 节关于指令吞吐量受限于访存延迟的分析。在向量化状态下，处理器依然有近 60% 的时钟周期处于无法派发微指令 (μops) 的停顿状态。
- **物理意义**：虽然 IPC 维持在 1.722，但由于 *Memory Bound* (34.3%) 的主导作用，执行端口（Execution Ports）大部分时间处于等待内存控制器（IMC）返回数据的空转状态，验证了访存延迟是影响点乘算法性能的主要因素。

6.5.2 L2 缓存缺失与延迟占比验证

- **观测数据**：E-core 采样显示 L2 Bound 为 34.6%，且 L2 Miss Rate 高达 65.4%。
- **结论印证**：该项数据支持了 6.3 节关于存储层级溢出与 TLB 穿透的论述。
- **因果分析**：由于 $N = 10^8$ 的数据规模超过 L2 缓存物理容量，高频的 L2 缺失迫使访存请求跨越核心边界向 L3 或 DRAM 发起访问。这种大规模的缓存失效增加了触发硬件页表漫游 (*Page Walk*) 的概率，从而产生了可观测的寻址惩罚延迟。

6.5.3 主存读写带宽利用率验证

- **观测数据**：实测 *Average DRAM Bandwidth* 达到 27.024 GB/s。
- **结论印证**：对于 Intel Core Ultra 7 硬件平台，27 GB/s 的持续带宽利用率反映了乱序执行引擎 (OoO) 对 MLP（内存并行度）的有效利用。
- **物理意义**：点乘算法的数据无依赖特性允许 CPU 同时维护多个在途 (In-flight) Load 请求。该吞吐量证明 OoO 引擎正在通过并发请求尝试掩盖主存延迟，印证了 6.4 节关于并行访存的分析。

6.6 总结：硬件瓶颈特征

- **指标权重的综合评价**：单纯的 IPC 或退休率无法全面表征程序执行质量，*Instructions per FLOP* 与 *Vectorization Rate* 的协同是衡量大规模数值计算效率的关键参考。
- **瓶颈位移规律**：优化过程实质上是将性能瓶颈从软件层面的寄存器资源分配缺失，转移至硬件层面的存储子系统延迟与带宽限制。
- **计算密度的制约**：点乘算法较低的计算密度 (*Arithmetic Intensity*) 决定了其执行过程偏向于 *Memory-Bound* 状态。在当前微架构下，程序的最终性能主要取决于处理器的内存级并行度 (MLP) 与主存子系统的吞吐能力。

7 Java C2 编译器机器码生成验证

为了观测 JVM 运行时的指令执行特征，本研究构建了基于底层反馈的诊断环境，直接输出和分析即时编译器生成的机器码质量。

7.1 反汇编诊断工具链配置

- **硬件环境**：**Intel Core Ultra 7 155H (Meteor Lake)**。
- **软件工具**：
  - **hsdis-amd64.dll**：硬件平台反汇编插件，部署于 `$JAVA_HOME/bin/server`，用于将 JVM 生成的二进制机器码翻译为 x86-64 汇编指令。
  - **核心命令**：

Listing 1: 开启诊断模式并导出汇编日志

```
# PowerShell 环境
java -XX:+UnlockDiagnosticVMOptions `
     -XX:+PrintAssembly `
     -XX:CompileCommand=compileonly,DotProduct::test* `
     DotProduct int 100000000 > jit_output.txt 2>&1
```

7.2 Sea of Nodes 中间表示与寄存器分配实证

Java C2 编译器通过构建 **Sea of Nodes** 图结构进行中间表示优化。在 `jitoutput.txt` 中，可观察到变量从操作数栈向物理寄存器映射的执行特征。

7.2.1 变量到物理寄存器的直接映射验证

在反汇编日志的内层循环片段中，算术指令流呈现以下结构：

Listing 2: 寄存器锁定汇编片段

```
; 地址: 0x0000028fbacf06ba
imul %r9d, %r8d  ; 将 v1[k](%r9d) 与 v2[k](%r8d) 相乘
add  %edi, %r8d  ; 累加结果至寄存器 %r8d
inc  %esi        ; 循环索引 k++ (%esi)
```

- **分析**：在未优化字节码中依赖 `iload` 和 `istore` 完成的操作，被 C2 编译器优化为**寄存器间（Register-to-Register）**的寻址。这证明 C2 能够将累加器变量分配至物理寄存器中，有效减少了对内存栈帧的重复访问。

7.3 Safepoint 轮询机制的运行时指令开销

由于托管语言的架构特性，Java 在生成算术指令的同时，会保留用于维护系统一致性的辅助状态检查指令。

7.3.1 辅助指令的运行时表现

在循环体的回边（Backedge）处，可观测到标记为 `{poll}` 的指令：

Listing 3: 安全点轮询指令

```
; 地址: 0x0000028fbacf06f6
test %eax, (%r10) ; {poll}
```

- **性能影响**：该指令不参与点乘逻辑，其作用是探测内存页状态以判断是否需要进入安全点（如触发 GC）。在十亿次迭代的循环结构中，该探测指令亦被等量执行。
- **结论**：这是 Java 运行时与原生 C 语言 (`-O3`) 在指令组成上的结构性差异。C 语言的生成代码中不包含此类用于环境状态维护的探针指令。

7.4 短位宽类型 (short) 的隐式转换指令激增现象

实测数据中，`short` 类型（70.20 ms）执行耗时高于 `int` 类型（48.26 ms）的现象，可通过对比两者在 C2 阶段生成的汇编指令密度进行解释。

- **结论印证**：基于 JVM 规范，小位宽整数运算依托 `int` 操作数栈槽位进行。因此，`short` 类型在计算过程中引入了**符号扩展**与**强制截断**的额外指令，增加了整体的计算周期开销。

7.5 本章小结：JVM 托管执行引擎的物理约束

通过对底层反汇编指令的解析，可以总结出 Java 虚拟机在执行密集型计算时的体系结构特征：

1. **JIT 编译器的图优化效果**：通过 Sea of Nodes 等中间表示，Java 在高频执行路径上能够实现与静态编译语言相近的物理寄存器分配策略。

表 6: `int` 与 `short` 核心循环指令流对比分析

阶段	testInt (32-bit Native)	testShort (16-bit to 32-bit)	性能影响分析
加载 (Load)	`movl` (直接加载 32 位)	`movswl` (带符号扩展加载)	`short` 需进行符号扩展，增加指令周期
计算 (Math)	`imull` (32 位乘法)	`imull` (32 位乘法)	计算模块位宽需求一致
截断 (Cast)	**(无)**	`movswl` / `i2s` (结果强制截断)	`short` 需额外指令处理 16 位溢出截断
依赖链	逻辑路径简短	存在隐式数据转换依赖	增加了 **RAW (Read-After-Write)** 延迟
指令密度	约 5–7 条核心指令	**约 8–11 条核心指令**	循环内指令总数增加，影响单位时间计算产出

2. **托管环境的结构性开销**：Safepoint 轮询等运行时辅助指令不可避免地占据了部分流水线资源，构成了托管语言特有的执行开销。
3. **数据类型处理规范的影响**：在 JVM 环境下，`int` 类型具备最高的算术执行效率。选用 `short` 或 `byte` 虽可降低内存占用空间，但受限于类型提升规范，其指令开销显著增加，在纯计算密集型场景下并非性能最优解。

8 全文总结与心得

8.1 全文总结

本项目从实现 C 语言与 Java 的多数据类型向量点乘算法入手，通过指数级规模的性能基准测试验证了时间复杂度的线性特征，并精准识别出受硬件缓存层级制约的“内存墙”瓶颈。在调试阶段，工作围绕 GCC 编译优化组合与 JVM 运行时堆参数展开，系统性对比了指令调度、自动向量化及分层编译对执行效率的影响。随后，实验通过 Intel VTune 采样深入至微架构层面，解构了流水线停顿、Backend Bound 及 TLB 穿透等物理约束，并最终借助反汇编工具抓取 JIT 机器码，实证了寄存器分配、Safepoint 轮询与隐式类型提升对计算密度的影响，完成了一次从高级语义到硅片底层逻辑的全链路性能剖析。

8.2 心路历程

拿到这个 project，向量点积，一次乘法一次加法，写起来是很逻辑清晰的，代码量也不大 (但是有对每个函数每种类型做参数检查)。编译选项千千万，算上 java 大概有 20 个，5 种数据类型，每一个类型从 1 到 1000000000 算九次，一次实验就能得到 1000 条左右的数据。测了 5 次取平均，但是这只是停留在数字而已，原来以为 java 慢，但是发现它竟然在大数据量面前可以跟 O3 差不多，这引起了我的兴趣。我认为这个项目对我来说更是一个学习型项目，我有很多知识不懂，但我希望从这个项目中了解一些原理。一开始我无从下手，在 AI 和博客的帮助下，我逐渐了解到了原理，但是看了原理之后我又会想问：真的如此吗？我不是 CPU，我也不是晶体管，我没有亲历这个过程，那我怎么知道 AI 说的对不对呢？于是，我的实验的后半段重心放在了原理的探究上，由于我是在 Windows 的本地跑的项目，使用的工具是 Intel Vtune，让我得以从复杂的结构中窥见一些脉络。原来，从代码到结果，中间的路非常漫长。实验想

做充足是很花时间和功夫的，而且我们难以将性能的进步简单的归因到某一个开关或者优化，不过这个 project 让我深深意识到 CPU 的强大，心存敬畏之心，下一个 projct 又布置了。